

PRODUCT REQUIREMENTS DOCUMENT (PRD) — ORTHOGRAPHIC QUADRANT RESTRUCTURING & VERTICAL INVERSION FIX

Document Version: 2.1 — Coordinate Engine & Quadrant Alignment Patch

Target Modules: src/engine/cad_engine.py, src/reconstruction/reconstructor.py,
src/reconstruction/brep_reconstructor.py, src/ui/canvas.py

Status: Approved for Implementation

1. Executive Summary & Root Cause Analysis

1.1 The Visual Inversion Bug

When a user draws a base chassis in Front View with wheels positioned at the lower boundary, the reconstructed 3D model places the wheels at the top of the box.

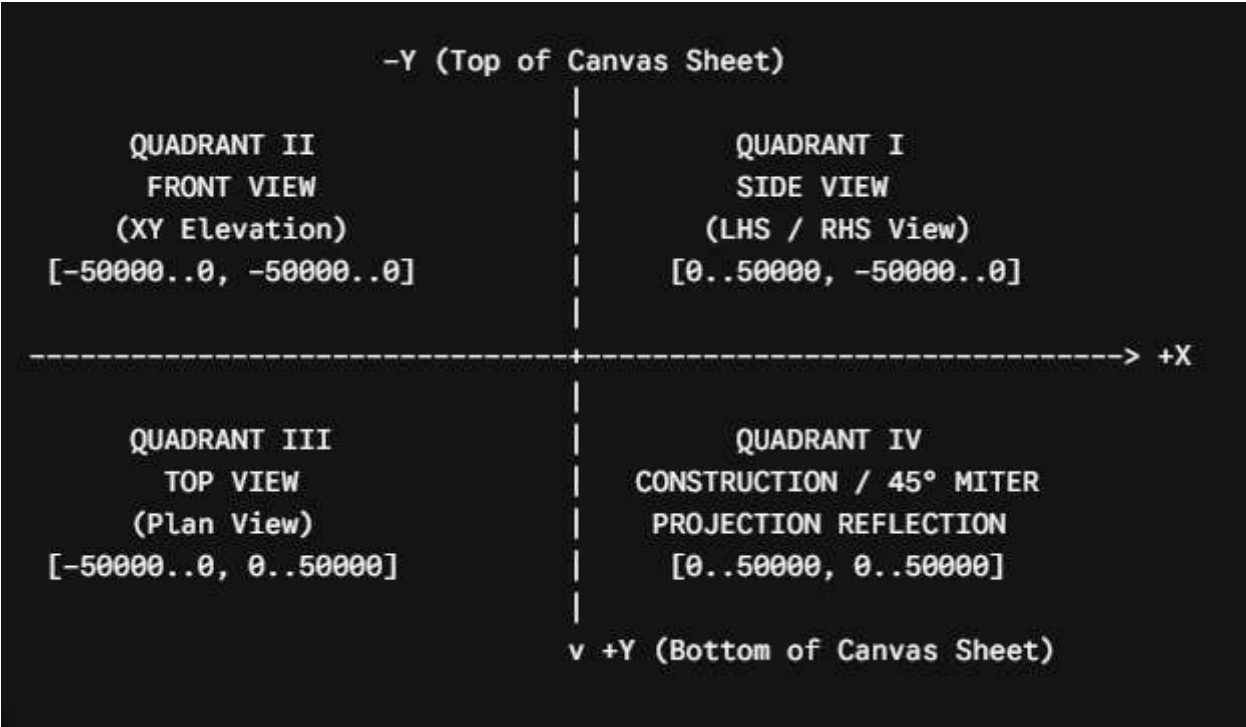
1.2 Mathematical Root Cause

In Qt QGraphicsScene, the vertical axis (Y_{canvas}) increases **downward**. In 3D Cartesian space, the vertical axis (Y_{3D}) increases **upward**.

1. **Origin Misalignment:** When Front View was positioned in the upper canvas quadrant, shapes drawn near the canvas origin $(0, 0)$ were interpreted as $Y_{\text{local}} = 0$ (the 3D ground level). Shapes drawn further down the canvas ($+Y_{\text{canvas}}$) were transformed to $+Y_{\text{3D}}$ (higher up in 3D space) instead of $-Y_{\text{3D}}$, effectively flipping the object upside down.
2. **First-Angle vs. Third-Angle Quadrant Drift:** Front View placed in Quadrant II without a matched bottom-origin inversion formula inverted the vertical sign before extrusion.

2. Updated Standard Quadrant Layout Specification

The 2D infinite drafting sheet is reorganized into the standard First-Angle / European industrial layout where Front View sits in Quadrant II, Side View sits in Quadrant I, and Top View sits in Quadrant III:



Quadrant Allocation & Bounds Table

Quadrant	Canvas Bounding Extents	Assigned Orthographic View	3D Plane Mapped	Local Origin (X0,Y0)
Quadrant I (Top-Right)	$X \in [0, \infty), Y \in (-\infty, 0]$ MD	Side View (LHS/RHS) MD	Z – Y Plane (Profile) MD	(0.0, 0.0) (Center Crosshair) MD
Quadrant II (Top-Left)	$X \in (-\infty, 0], Y \in (-\infty, 0]$ MD	Front View	X – Y Plane (Elevation) MD	(0.0, 0.0) (Center Crosshair) MD
Quadrant III (Bottom-Left)	$X \in (-\infty, 0], Y \in [0, \infty)$ MD	Top View	X – Z Plane (Plan) MD	(0.0, 0.0) (Center Crosshair) MD
Quadrant IV (Bottom-Right)	$X \in [0, \infty), Y \in [0, \infty)$ MD	45° Miter Guide	Projection Transfer MD	(0.0, 0.0) (Center Crosshair) MD

3. Mathematical Coordinate & Extrusion Transformations

3.1 2D Canvas Point to Local View Inversion (u, v)

All quadrants now project from the central axis intersection $(0, 0)$ as the global baseline datum:

1. Front View (Quadrant II: $X \leq 0, Y \leq 0$)

- Horizontal axis: $u = p_x - 0 = p_x$ (or normalized width offset from minimum boundary)
- Vertical axis (Canvas Y up is negative; 3D Y up is positive):

$$v = -p_y$$

Result: Points near the horizontal dividing axis ($p_y \approx 0$) sit at the bottom ($v = 0$). Points extending up into Quadrant II (negative p_y , e.g., $p_y = -100$) yield $v = -(-100) = +100$ (top of the body). Circles drawn at $p_y = -20$ sit at $v = 20$, correctly placing them below the main body at $v \in [20, 100]$.

2. Side View (Quadrant I: $X \geq 0, Y \leq 0$)

- Depth axis (LHS): $u = p_x$
- Vertical axis:
 $v = -p_y$
Result: Side view height perfectly matches Front View height across the horizontal datum line $Y = 0$.

3. Top View (Quadrant III: $X \leq 0, Y \geq 0$)

- Horizontal axis (Width): $u = p_x$
- Depth axis (Extending down into Quadrant III represents increasing depth away from front):
 $w = p_y$

3.2 4x4 Homogeneous Extrusion Matrices (M)

Extruded 2D local profiles (u, v, t) with profile thickness parameter t are transformed into 3D world coordinates (X, Y, Z) :

$$\begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix} = M \begin{pmatrix} u \\ v \\ t \\ 1 \end{pmatrix}$$

1. Front View Extrusion Matrix (M_{front})

Front sketches lie directly in the X - Y plane and extrude along $+Z$ (Depth):

$$M_{\text{front}} = \begin{pmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & Z_{\min} \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$$

2. Side View Extrusion Matrix (M_{side})

Side sketches lie in the Z - Y plane and extrude along $+X$ (Width):

$$M_{\text{side}} = \begin{pmatrix} 0.0 & 0.0 & 1.0 & X_{\min} \\ 0.0 & 1.0 & 0.0 & 0.0 \\ 1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$$

3. Top View Extrusion Matrix (M_{top})

Top sketches lie in the X - Z plane and extrude vertically along $+Y$ (Height):

$$M_{\text{top}} = \begin{pmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & Y_{\min} \\ 0.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{pmatrix}$$

3.3 Relocated 45° Miter Line Geometry (Quadrant IV)

With Top View in Quadrant III ($X \leq 0, Y \geq 0$) and Side View in Quadrant I ($X \geq 0, Y \leq 0$), the 45° projection transfer takes place in **Quadrant IV** ($X \geq 0, Y \geq 0$):

- **Origin:** Canvas center $(0.0, 0.0)$

- **Ray Equation:**

$$\text{Ray}(t) = (+t, +t) \quad \text{for } t \geq 0$$

- **Projection Reflection Math:**

$$Y_{\text{top_proj}} = X_{\text{side_proj}}$$

- Any vertical displacement down into Top View ($+Y_{\text{top}}$) reflects off the 45° line ($Y = X$) into positive horizontal displacement ($+X_{\text{side}}$) in the Side View.

4. Module-by-Module Technical Implementation

4.1 src/engine/cad_engine.py

1. Update Default Quadrant Boundaries in `_init_default_regions()`:

- front: bounds = (-50000.0, -50000.0, 0.0, 0.0)
- side / left_side: bounds = (0.0, -50000.0, 50000.0, 0.0)
- top: bounds = (-50000.0, 0.0, 0.0, 50000.0)

2. Update Centroid Routing (`assign_shape_to_region`):

- If $C_x \leq 0$ and $C_y \leq 0$ implies `'front'`
- If $C_x \geq 0$ and $C_y \leq 0$ implies `'side'`
- If $C_x \leq 0$ and $C_y \geq 0$ implies `'top'`

3. Local Coordinate Normalization:

- Set Front View $v = -p_y$ and Side View $v = -p_y$.

4.2 src/ui/canvas.py

1. Quadrant Visual Overlay Labels & Colors:

- Quadrant II (Top-Left): Label "**Front View (XY Plane)**" (Cyan / Blue accent)
- Quadrant I (Top-Right): Label "**Side View (ZY Plane)**" (Amber / Orange accent)
- Quadrant III (Bottom-Left): Label "**Top View (XZ Plane)**" (Green accent)
- Quadrant IV (Bottom-Right): Label "**45° Miter Guide**" (Faint dashed grey / yellow)

2. Miter Line Draw Function (`draw_miter_line`):

- Update start point to (0, 0) and end point to (50000, 50000) extending down-right into Quadrant IV.

3. Cross-View Alignment Guides (`_sync_projection_guides`):

- Moving the cursor in Front View (Q2) projects horizontal alignment rays across into Side View (Q1) and vertical alignment rays down into Top View (Q3).

4.3 src/reconstruction/reconstructor.py & `brep_reconstructor.py`

1. Profile Polygon Transformation:

- Convert Front View Shapely primitives using $y_{\text{local}} = -y_{\text{canvas}}$.
- Convert Side View Shapely primitives using $y_{\text{local}} = -y_{\text{canvas}}$.
- Convert Top View Shapely primitives using $z_{\text{local}} = y_{\text{canvas}}$.

2. CSG Intersection Pipeline:

- Apply updated transformation matrices M_{front} , M_{side} , and M_{top} prior to running `trimesh.boolean.intersection()` and `build123d B-Rep intersection`.

3. Orientation & Normal Safeguards:

- Verify $\text{mesh.volume} > 0$ and enforce outward face normal orientation with `trimesh.repair.fix_normals()`.

5. Verification Plan & Test Cases

5.1 Automated Unit Tests (`tests/test_quadrant_inversion_fix.py`)

Python code:

```
import pytest
from src.engine.cad_engine import CADEngine, Rectangle, Circle
from src.reconstruction.reconstructor import Reconstructor3D

def test_front_view_bottom_wheels_orientation():
    engine = CADEngine()

    # 1. Add Main Body to Front View (Q2: X in [-100, -20], Y in [-100, -40])
    # Local Height: v in [40, 100]
    body = Rectangle(rect=(-100.0, -100.0, 80.0, 60.0), layer="Visible")
    engine.add_shape(body, view="front")

    # 2. Add Wheels to Bottom of Front View (Q2: Y in [-40, -20])
    # Local Height: v in [20, 40] -> Strictly below the body
    wheel_left = Circle(center=(-80.0, -30.0), radius=10.0, layer="Visible")
    wheel_right = Circle(center=(-40.0, -30.0), radius=10.0, layer="Visible")
    engine.add_shape(wheel_left, view="front")
    engine.add_shape(wheel_right, view="front")

    # 3. Add Side View Depth Extrusion (Q1: X in [20, 40], Y in [-100, -20])
    side_body = Rectangle(rect=(20.0, -100.0, 20.0, 80.0), layer="Visible")
    engine.add_shape(side_body, view="side")

    # 4. Reconstruct 3D CSG Solid
    reconstructor = Reconstructor3D()
    mesh = reconstructor.reconstruct(engine.shapes)

    # 5. Assert Vertical Coordinates in 3D Space
    # Body Y bounds must be strictly higher than Wheel center Y coordinates
    wheel_left_3d_y = -(-30.0) # Expected 3D Y = 30.0
    body_top_3d_y = -(-100.0) # Expected 3D Y = 100.0
```

```
assert mesh is not None
assert mesh.bounds[1][1] > mesh.bounds[0][1] # Valid height
assert mesh.bounds[0][1] <= 20.0 + 1e-3    # Lowest point is bottom of wheels
assert mesh.bounds[1][1] >= 100.0 - 1e-3    # Highest point is top of body
```

5.2 Visual & Manual Acceptance Criteria

1. Re-create the chassis with bottom wheels shown in the reference image.
2. In the 3D viewport, verify that the wheels rest on the bottom plane with the main chassis body and cutouts resting above them.
3. Draw in Quadrant II (Front) and verify that horizontal projection lines align with Quadrant I (Side) and vertical projection lines align with Quadrant III (Top).
4. Export the resulting solid to STEP format and verify that opening the model in FreeCAD/SolidWorks displays positive $+Y$ as the vertical upright axis.